

Transformer-based sequence modelling for the Cora citation graph

Abstract

Graph data contains entities and their relationships, but it does not always have to be processed directly with a graph neural network. A graph can also be sampled as a collection of random walks and represented as sequences. This report investigates whether Transformer-style sequence models can learn predictive relationships from such sequences on the Cora citation graph.

The study uses four related tasks. First, a BERT-style encoder predicts the next node label from labels observed earlier in a random walk. Second, a sequence-to-sequence Transformer predicts future Cora node-feature vectors from past node features. These features are the 1,433-dimensional sparse word-based vectors supplied by the dataset; the current implementation does not construct additional centrality or community features. Third, another sequence-to-sequence model predicts future node labels from past node features. Finally, Experiment 4, presented as a strict node-split appendix, restricts random walks to Cora’s official training, validation, and test masks. This separates ordinary walk-level evaluation from a stricter setting in which generated walks are restricted to disjoint node partitions.

The main experiments use random-walk splits and therefore measure generalisation across sampled walks rather than strict node-disjoint generalisation. The appendix provides a stricter node-partition diagnostic, although the underlying graph remains the full Cora graph. The models use randomly initialised BERT-style Transformer components rather than a language-pretrained BERT checkpoint. The completed results are 0.8113 next-label accuracy, 0.011614 future-feature MSE, 0.2931 feature cosine similarity, 0.4675 nearest-neighbour label retrieval accuracy, and 0.7676 generated future-label token accuracy in the main walk-level setting. Under Experiment 4, the strict node-split evaluation, generated token accuracy fell to 0.2251 and generated sequence accuracy was 0.0971.

Overall, the models learn useful walk-level label regularities, but performance drops sharply when the official Cora node partitions are kept separate.

1 Introduction

Graphs are a natural representation for many datasets. Nodes can represent people, documents, or scientific papers, while edges represent relationships such as friendship, citation, or communication. A graph neural network usually processes this structure through message passing: each node repeatedly aggregates information from its neighbours and updates its representation. This is effective for local relational patterns, but increasing the number of message-passing layers can make optimisation and representation quality more difficult. Deep graph models can also become expensive when the graph is large or when useful dependencies extend beyond a small neighbourhood.

Transformers provide a different way to model dependencies. Self-attention lets each position in an input sequence interact directly with other positions in the same sequence. If graph walks are represented as sequences, a Transformer can process a sampled path in a way that resembles language modelling. It provides a sequence-based view of graph information that can be tested independently from message passing.

This report applies that idea to Cora, a citation-network benchmark containing scientific publications represented by sparse word features and class labels. Random walks over the citation graph are used as graph sequences. The experiment is deliberately divided into separate prediction tasks because predicting a future feature vector and predicting a future class label test different properties of the model.

1.1 Motivation

The present experiment is motivated by the GNN practicum project in which Transformer-based sequence modelling was applied to graph walks. That earlier experiment used engineered centrality and community features to predict node labels within a graph sequence. The current study applies the related graph-sequence idea to the Cora citation graph, but it uses a different graph, a different feature representation, and different prediction tasks. It should therefore be understood as a separate application rather than a direct continuation or replication of the practicum experiment.

Cora contains scientific publications as nodes, citation relationships as edges, sparse word-based feature vectors, and class labels. The current implementation uses Cora’s original 1,433-dimensional node-feature matrix and does not construct additional centrality or community features. This makes it possible to examine how Transformer-style sequence models behave when applied to a scientific citation graph with native word-based features.

In the previous experiment, the encoder received the feature sequence for the entire walk, including the feature vectors of nodes whose labels were being predicted. Schematically, if nodes C and D were target positions, the setup included:

`features(A), features(B), features(C), features(D) -> labels(C), labels(D)`

Consequently, the feature vectors of the target nodes were already available to the encoder when their labels were predicted. The current Cora experiment changes this setup. In RQ3, the encoder receives only the feature vectors of the preceding nodes:

`x1, x2, x3, x4 -> y5, y6, y7, y8`

The future-node feature vectors x_5, x_6, x_7, x_8 are withheld from the encoder. RQ3 therefore tests whether features observed earlier in a random walk can help predict the labels of later nodes without directly exposing the target nodes’ own features.

Predicting future labels does not show whether the model can reconstruct the feature information associated with future nodes. A model may learn class-related patterns without accurately forecasting the future nodes’ feature vectors. The study therefore includes RQ2 as a separate future-feature forecasting task:

`x1, x2, x3, x4 -> x5, x6, x7, x8`

Both sequence-to-sequence models use teacher forcing during training. The decoder receives the true previous future target while predicting the next target. During generated evaluation, it receives its own previous prediction instead.

RQ4 was added to determine whether the RQ3 future-label results remain valid when node partitions are kept strictly separate. In this setting, training, validation, and test random walks are generated separately within Cora’s official training, validation, and test node masks. The generated walks are restricted to their respective masks and therefore cannot mix nodes from different partitions. The underlying graph remains the full Cora graph. Ordinary evaluation measures performance on randomly divided graph walks, whereas strict evaluation provides a more demanding node-partition diagnostic for nodes that were not present in the training walks.

1.2 Research questions

The study investigates four related questions:

- RQ1: Can a Transformer encoder predict the label of the next node in a random walk from the labels of the preceding nodes?
- RQ2: Can a sequence-to-sequence Transformer predict the feature vectors of future nodes from feature vectors observed earlier in the random walk?
- RQ3: Can a sequence-to-sequence Transformer predict the labels of future nodes when its encoder receives only the feature vectors of preceding nodes?
- RQ4: How does the RQ3 future-label task change when random walks are restricted to Cora’s official training, validation, and test node masks?

RQ1 is a label-transition baseline. RQ2 tests future-feature forecasting. RQ3 tests feature-conditioned future-label forecasting. RQ4 tests whether the RQ3 conclusions remain when node partitions are kept separate.

1.3 Structure of the report

Section 2 documents the use of generative AI. Section 3 introduces graph sequence modelling, random walks, Transformers, and the Cora dataset. Section 4 describes the data preparation, model architectures, training procedure, and evaluation protocol. Section 5 records the experimental results. Section 6 discusses the results and the difference between walk-level and strict node-split evaluation. Sections 7 to 9 give the conclusion, limitations, and possible future work.

2 Use of generative AI

Generative AI was used to help with the following parts of the work:

1. organising the report structure;
2. drafting and revising explanations of the models and experiments;
3. checking the consistency of the experimental questions and evaluation protocol;
4. identifying possible label and node leakage in an earlier implementation;
5. improving the wording and formatting of the report;

6. helping interpret results after the numerical outputs were available.

All generated text and code were checked against the implementation and the experiment logs. The author remains responsible for the correctness of the final report, the validity of the evaluation protocol, and the interpretation of the results. Numerical values were copied from executed outputs rather than estimated from expected behaviour.

3 Background

3.1 Graph neural networks and long-range information

A graph consists of a set of nodes and a set of edges connecting related nodes. In a message-passing graph neural network, a node representation is updated using information from neighbouring nodes. Repeating this operation allows information to travel farther through the graph, but deeper networks can be harder to optimise and may produce overly similar node representations. These issues motivate experiments with alternative representations of graph structure.

The present study tests the narrower question of whether a Transformer can learn useful patterns when the graph is sampled into random-walk sequences.

3.2 Random walks as graph sequences

A random walk starts at a graph node and repeatedly selects one of its neighbours. The resulting sequence records a local path through the graph. Following the sequence-learning view used in methods such as DeepWalk, node labels or node features can be treated as observations at successive positions.

A walk is not a deterministic description of the graph. The same node can lead to several possible next nodes, and two walks can overlap heavily. This matters for evaluation. A random split of walks can place the same physical node in training and test walks. Such a split can still be useful for measuring prediction on new sampled paths, but it must not be presented as strict unseen-node generalisation.

3.3 Transformer and BERT-style encoders

3.3.1 Transformer architecture

A Transformer is a neural-network architecture designed to process ordered sequences. Unlike a recurrent neural network, it does not process a sequence strictly one element at a time. Instead, it allows all positions in a sequence to exchange information through self-attention. This makes it possible for a position near the end of a sequence to use information from an earlier position directly, without passing that information through a long chain of recurrent updates.

For a graph walk, each position corresponds to a node in the sampled sequence. Depending on the experiment, the input at a position can be a one-hot node label or a numerical feature vector. The Transformer first maps these inputs into a common hidden representation. If the original input dimension is d_{in} and the Transformer hidden dimension is d_{model} , a learned projection converts each input vector into a hidden vector:

$$\text{input vector} \longrightarrow \text{linear projection} \longrightarrow \text{hidden representation.}$$

The central operation is self-attention. Given a sequence of hidden vectors H , the model creates three representations: queries Q , keys K , and values V . The attention scores are computed by comparing each query with every key:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

A learned position representation is added to each input so that the model can distinguish the first, second, third, and fourth positions in the walk. Position information is important in this project because the prediction target depends on the direction of the sequence. The task is not simply to recognise which nodes appear in a walk; it is to predict labels or features at later positions from earlier positions.

The original Transformer architecture contains two main components:

- an encoder, which builds contextual representations of the input sequence;
- a decoder, which generates an output sequence using the encoder representations.

The decoder normally contains both self-attention over the partially generated output and cross-attention to the encoder output. A causal mask is used in the decoder so that a position cannot access future output tokens during generation.

The sequence-to-sequence models in this project follow this general encoder-decoder design. Their encoders process the four past node-feature vectors, while their decoders generate the four future feature vectors or future node labels. During training, teacher forcing supplies the previous correct target to the decoder. During generated evaluation, the decoder must instead use its own previous prediction.

3.3.2 How BERT differs from a general Transformer

BERT stands for Bidirectional Encoder Representations from Transformers. It is based on the Transformer encoder, but it is not the complete original encoder-decoder Transformer architecture. BERT is encoder-only. Its self-attention is bidirectional: each input position can attend to other positions on both sides of it. BERT does not use a causal mask in its encoder because it is not generating the input sequence one position at a time. It reads the available input sequence as a whole and produces a contextual representation for every position. The present project does not use that language-pretraining procedure. The models are created with `BertConfig` and `BertModel`, but no pretrained checkpoint is loaded. They are trained from random initialisation on the Cora graph data.

3.4 Sequence-to-sequence prediction

A sequence-to-sequence model maps one sequence to another. It contains an encoder, which represents the observed input sequence, and a decoder, which produces the target sequence. The encoder produces a sequence of contextual hidden representations, often called the encoder memory. The decoder uses two types of attention:

- self-attention over the decoder inputs;
- cross-attention to the encoder memory.

Decoder self-attention allows the decoder to use the previous target values that have already been generated or supplied during training. Cross-attention allows each decoder position to retrieve relevant information from the encoded past-node features. During training, the decoder uses teacher forcing. This means that the decoder receives the correct previous target rather than its own previous prediction. The decoder input is shifted by one position so that the current target is never supplied before it is predicted.

3.5 Cora citation dataset

Cora is a commonly used citation-network benchmark for node classification. It contains 2,708 scientific publications, represented as nodes in a graph. Each publication belongs to one of seven research-topic classes: Case-Based, Genetic Algorithms, Neural Networks, Probabilistic Methods, Reinforcement Learning, Rule Learning, or Theory.

The original Cora citation data is commonly reported as containing 5,429 citation links. However, the PyTorch Geometric Planetoid loader stores graph connectivity in the `edge_index` format. In the loaded representation used by this experiment, `data.edge_index` contains 10,556 source-destination entries. These two numbers describe different storage representations and should not be treated as identical edge counts.

Each publication is represented by a 1,433-dimensional binary bag-of-words feature vector. Each dimension corresponds to a word in the dataset vocabulary. A value of one indicates that the corresponding word is present in the publication’s feature representation, while a value of zero indicates that it is absent. The model receives these numerical feature vectors rather than the original publication text.

The PyTorch Geometric loader also provides an integer label for every publication and the standard training, validation, and test node masks. In the implementation, labels are encoded as integer class IDs from 0 to 6. The script displays these labels using generic names such as `class_0` and `class_1`; these names are aliases created for the experiment and do not replace the underlying Cora topic categories. Because the exact integer-to-topic mapping was not independently verified from the loaded metadata for this run, the reported class-wise metrics retain the generic class names.

4 Methodology

4.1 Reproducibility and configuration

The experiment sets the Python, NumPy, and PyTorch random seeds to 42. CUDA is used when available; otherwise the models run on the CPU. The main sequence parameters are:

The exact device and dataset sizes are printed by the script at runtime.

4.2 Random-walk generation and main split

Five random walks are generated from each node using the undirected NetworkX graph. Each walk has eight nodes when a complete walk can be generated. The walk list is divided into:

- 70% training walks;
- 15% validation walks;

Parameter	Value
Random seed	42
Walk length	8
Past length	4
Future length	4
Main walks per node	5
Batch size for feature/label sequence tasks	32
Batch size for next-label task	64
Next-label epochs	10
Sequence-model maximum epochs	30
Early-stopping patience	5
Learning rate	0.0001

- 15% test walks.

The split uses a fixed random state of 42. The split is performed at the walk level. It is therefore possible for a physical node to occur in walks belonging to more than one split. This is an intentional distinction in this report: the main results measure walk-level performance, while Experiment 4 addresses node-partition separation.

4.3 Data preparation for Experiment 1

Each eight-node walk is represented as $n_1, n_2, \dots, n_8$, where y_i denotes the class label of node n_i . Experiment 1 performs next-label prediction using a sliding window of four previous labels, creating examples such as:

y1, y2, y3, y4 -> y5
y2, y3, y4, y5 -> y6
y3, y4, y5, y6 -> y7
y4, y5, y6, y7 -> y8

The input labels are converted into one-hot vectors, while the target remains an integer class ID.

4.4 Data preparation for Experiments 2 and 3

Each eight-node walk is divided into two sections: an observed past prefix containing the first four nodes, n_1, n_2, n_3, n_4 , and a future segment containing the final four nodes, n_5, n_6, n_7, n_8 . The corresponding feature and label sequences are x_1, x_2, x_3, x_4 and y_1, y_2, y_3, y_4 for the past, followed by x_5, x_6, x_7, x_8 and y_5, y_6, y_7, y_8 for the future.

Here, x_i represents the 1,433-dimensional feature vector of node n_i , and y_i represents its class label. Experiment 2 predicts future features:

encoder input: x1, x2, x3, x4
target sequence: x5, x6, x7, x8

Experiment 3 predicts future labels from the same observed feature prefix:

encoder input: x1, x2, x3, x4
target sequence: y5, y6, y7, y8

Therefore, Experiments 2 and 3 use the same encoder input but represent different prediction tasks, output spaces, and loss functions. During teacher-forced training, the feature decoder receives a beginning zero vector followed by the previous true feature vectors:

zero vector, x_5 , x_6 , x_7 -> predict x_5 , x_6 , x_7 , x_8

The label decoder receives a beginning-of-sequence symbol followed by the previous true labels:

BOS, y_5 , y_6 , y_7 -> predict y_5 , y_6 , y_7 , y_8

During generated evaluation, both decoders instead receive their own previous predictions. The future feature vectors x_5 through x_8 are not supplied to the encoder when predicting the corresponding labels y_5 through y_8 . Although these feature vectors are not labels themselves, they describe the same future nodes and could provide information about their labels. Excluding them prevents target-feature exposure and ensures that Experiment 3 measures prediction of future labels from the past feature prefix alone.

4.5 Architecture for Experiment 1

The next-label model has three main parts:

1. a linear projection from the seven-dimensional one-hot label vector to a hidden size of 128;
2. a two-layer BERT-style encoder with four attention heads;
3. a linear classification layer that predicts one of the seven classes.

The output at the first input position is used for the next-label prediction. Training uses cross-entropy loss and AdamW optimisation. In the completed run, the script reported training loss over ten epochs and then evaluated 8,124 held-out examples; it did not print a validation-loss checkpoint selection for this experiment. The result should therefore be described as a fixed ten-epoch walk-level evaluation.

4.6 Architecture for Experiment 2

The future-feature model uses:

1. a linear projection from the 1,433-dimensional input feature vector to a hidden size of 128;
2. a two-layer BERT-style encoder with four attention heads;
3. a two-layer causal Transformer decoder;
4. a linear output layer returning a 1,433-dimensional feature vector.

The decoder is trained with teacher forcing. Its input is a zero vector followed by the true previous future features:

zero -> x_5 -> x_6 -> x_7

The model predicts:

x5, x6, x7, x8

The primary loss is mean squared error. Evaluation reports mean squared error and cosine similarity. An additional nearest-neighbour metric compares each predicted vector with all Cora node feature vectors and assigns the label of the closest vector. Because this candidate pool includes all nodes, including nodes in the test set, the metric is reported as a retrieval diagnostic rather than strict unseen-node classification.

4.7 Architecture for Experiment 3

The future-label model uses the same feature encoder and causal Transformer decoder pattern, but replaces the continuous feature output layer with a categorical classifier. Its decoder vocabulary contains the seven Cora labels plus a beginning-of-sequence token.

The task is:

x1, x2, x3, x4 -> y5, y6, y7, y8

Training uses cross-entropy loss with teacher forcing. Testing reports both teacher-forced token and sequence accuracy, and generated token and sequence accuracy using greedy autoregressive decoding. The generated scores are the more demanding evaluation because the decoder must use its own previous predictions.

4.8 Experiment 4: strict node-split evaluation

Experiment 4 uses the official Cora masks provided by PyTorch Geometric. Separate walks are generated inside the training, validation, and test node sets. When a walk is extended, only neighbours belonging to the same allowed mask may be selected.

The script tries several task lengths, beginning with:

x1, x2, x3, x4 -> y5, y6, y7, y8

and falling back to shorter configurations if the validation or test mask does not provide enough complete walks. The selected configuration and the resulting number of walks are printed before training.

This experiment prevents a walk from directly crossing from one official node partition into another. It does not solve every possible source of dependence: the graph itself is still derived from the full Cora network, and the task remains based on random walks. It does, however, provide a stricter node-partition comparison than the main walk-level split.

4.9 Evaluation measures

For categorical prediction, the report uses:

- cross-entropy loss;
- token accuracy;
- complete-sequence accuracy;
- precision, recall, and F1-score by class;

- confusion matrices where applicable.

For feature prediction, the report uses:

- mean squared error;
- cosine similarity;
- nearest-neighbour label accuracy as a separate retrieval diagnostic.

A complete sequence is counted as correct only when every predicted future position matches its target. Consequently, sequence accuracy can be much lower than token accuracy even when most individual predictions are correct.

5 Results and analysis

The following values were copied from the completed output for Experiments 1–3 and Experiment 4, the strict node-split evaluation.

5.1 Dataset and split summary

Quantity	Value
Nodes	2,708
Feature dimensions	1,433
Classes	7
edge_index entries	10,556
Device	CPU
Main random walks	13,540
Main training walks	9,478
Main validation walks	2,031
Main test walks	2,031
Future-prediction training examples	9,478
Future-prediction validation examples	2,031
Future-prediction test examples	2,031

The next-label task reported 37,912 training examples and 8,124 held-out test examples. Its output did not report a validation-example count or validation-loss checkpoint.

5.2 Experiment 1: past labels to next label

The model was trained for ten epochs. The training loss decreased from 0.6990 to 0.5685. On 8,124 held-out examples, it achieved:

The class-wise F1-scores were:

The majority class accounts for approximately 28.7% of the held-out labels, while uniform guessing over seven classes would give approximately 14.3%. The observed 81.13% accuracy is therefore well above these simple reference points. The strongest F1-score is

Metric	Test value
Accuracy	0.8113
Macro precision	0.79
Macro recall	0.80
Macro F1	0.79
Weighted F1	0.81

Class	Precision	Recall	F1	Support
<code>class_0</code>	0.70	0.69	0.69	1,149
<code>class_1</code>	0.75	0.76	0.75	662
<code>class_2</code>	0.87	0.91	0.89	1,267
<code>class_3</code>	0.85	0.85	0.85	2,332
<code>class_4</code>	0.89	0.82	0.85	1,317
<code>class_5</code>	0.79	0.74	0.77	889
<code>class_6</code>	0.69	0.82	0.75	508

for `class_2` (0.89). `class_0` and `class_6` are more difficult, with F1-scores of 0.69 and 0.75 respectively.

This is a walk-level next-label result. It indicates that label history in the sampled walks contains useful predictive information, but it does not establish strict unseen-node generalisation because the main split is performed over walks rather than disjoint node sets.

5.3 Experiment 2: past features to future features

The feature model’s validation MSE gradually improved during training, reaching 0.011537 at epoch 30. On the test set it achieved:

Metric	Test value
Mean squared error	0.011614
Cosine similarity	0.2931
Nearest-neighbour label accuracy	0.4675

The feature-reconstruction result is modest in terms of cosine similarity. The predicted feature vectors contain some information about the future vectors, but they are not close to the target vectors in a strong geometric sense. The nearest-neighbour label accuracy of 46.75% is above the majority-class reference of approximately 28.7%, but it must remain a diagnostic rather than a strict classification result because the retrieval candidate pool contains the full Cora feature matrix, including test nodes.

The validation nearest-neighbour scores were unstable across epochs, ranging from approximately 0.08 to 0.45, while MSE and cosine similarity changed more smoothly. The code tracked the best validation MSE and used it for early-stopping decisions, but it did not save and restore a best model state. Because the feature model continued through epoch 30, the reported test metrics correspond to the epoch-30 model. A future final run should explicitly restore the best checkpoint before testing.

5.4 Experiment 3: past features to future labels

5.4.1 Teacher-forced evaluation

Metric	Test value
Loss	0.5786
Token accuracy	0.8107
Complete-sequence accuracy	0.6169
Macro precision	0.79
Macro recall	0.80
Macro F1	0.79
Weighted F1	0.81

5.4.2 Generated evaluation

Metric	Test value
Generated token accuracy	0.7676
Generated sequence accuracy	0.6169
Macro precision	0.75
Macro recall	0.75
Macro F1	0.75
Weighted F1	0.77

Teacher-forced token accuracy was 81.07%, whereas generated token accuracy was 76.76%. This is a decrease of approximately 4.31 percentage points when the model must use its own previous predictions. The generated sequence accuracy was 61.69%. In this run, the teacher-forced and generated sequence accuracies were almost identical after rounding, even though their token-level scores differed.

The code used validation loss for early stopping but did not restore the best validation checkpoint. The best validation loss shown in the log was 0.5284 at epoch 15, while early stopping occurred after epoch 20. Consequently, the reported Experiment 3 test metrics are from the epoch-20 model rather than the best-validation epoch, and should be labelled accordingly.

The generated class-wise F1-scores were 0.64 for `class_0`, 0.67 for `class_1`, 0.86 for `class_2`, 0.81 for `class_3`, 0.83 for `class_4`, 0.71 for `class_5`, and 0.72 for `class_6`. The model performs best on the more frequent classes, particularly `class_2`, `class_3`, and `class_4`.

The result should not be described as direct feature-to-label reconstruction. The encoder receives only the first four feature vectors, while the target labels correspond to the following four nodes. The task therefore measures future-label prediction from a walk prefix, subject to the limitations of the walk-level split.

5.5 Experiment 4: strict node-split evaluation

The strict evaluation used Cora’s official masks:

Quantity	Value
Strict train nodes	140
Strict validation nodes	500
Strict test nodes	1,000
Selected task	x1 x2 x3 x4 -> y5 y6 y7 y8
Strict training walks	9,300
Strict validation walks	75,900
Strict test walks	207,600
Strict test tokens	830,400

The strict model overfit the small training partition quickly. Training loss fell from 0.3647 in epoch 1 to 0.0367 by epoch 6, while validation loss increased from 2.1251 to 2.9775. Early stopping occurred after epoch 6. The code did not restore the best validation checkpoint; the best strict validation loss was the epoch-1 value of 2.1251. The strict test metrics therefore correspond to the epoch-6 model, not to a restored best-validation model.

5.5.1 Teacher-forced strict evaluation

Metric	Test value
Loss	3.3809
Token accuracy	0.4793
Complete-sequence accuracy	0.0971
Macro precision	0.46
Macro recall	0.46
Macro F1	0.44
Weighted F1	0.49

5.5.2 Generated strict evaluation

Metric	Test value
Generated token accuracy	0.2251
Generated sequence accuracy	0.0971
Macro precision	0.42
Macro recall	0.30
Macro F1	0.20
Weighted F1	0.16

The generated strict class-wise scores were:

The strict test contains 830,400 target tokens. Its majority class accounts for approximately 31.67% of those tokens, while uniform guessing over seven classes gives approximately 14.29%. The teacher-forced score of 47.93% is above the majority baseline,

Class	Precision	Recall	F1	Support
<code>class_0</code>	0.14	0.31	0.19	103,858
<code>class_1</code>	0.26	0.83	0.40	71,823
<code>class_2</code>	0.87	0.07	0.12	128,582
<code>class_3</code>	1.00	0.02	0.04	262,999
<code>class_4</code>	0.25	0.46	0.32	127,778
<code>class_5</code>	0.24	0.03	0.05	80,366
<code>class_6</code>	0.20	0.36	0.25	54,994

but the generated score of 22.51% is below it and only 8.22 percentage points above uniform guessing.

The comparison with the ordinary walk-level result is:

Evaluation setting	Generated token accuracy	Generated sequence accuracy
Main walk-level split	0.7676	0.6169
Experiment 4 strict node-split evaluation	0.2251	0.0971

The strict generated token accuracy is approximately 54.25 percentage points lower than the main generated score, and strict sequence accuracy is approximately 51.98 percentage points lower. This is a substantial generalisation gap. The result indicates that the strong main-split performance depends on properties of the walk-level sampling setting and does not transfer reliably when the official node partitions are kept separate.

The strict confusion matrix also shows strong class-specific differences. Generated predictions have particularly low recall for `class_2`, `class_3`, and `class_5`, despite high precision for `class_3`. The model is therefore not simply making uniformly random errors; it is producing a biased and poorly calibrated distribution under the strict setting.

6 Discussion

The completed results support a mixed interpretation. Experiment 1 predicts the next label from the previous four labels and achieves 81.13% accuracy. Experiment 3 predicts four future labels from four past feature vectors and achieves 76.76% generated token accuracy. These results show that the sampled walks contain useful predictive information. For example, nearby nodes in the walk may often have related or repeated labels. However, the main split divides walks, not necessarily nodes. The same node may appear in both training and test walks. Therefore, the model may encounter familiar nodes, feature patterns, or local graph structures during testing. The results demonstrate performance on held-out walks, but they do not demonstrate that the model can classify completely unseen Cora nodes.

Experiment 2 predicts the feature vectors of future nodes from the feature vectors of the preceding nodes. The model achieved a test mean squared error of 0.011614 and a cosine similarity of 0.2931, but these values alone do not establish that the reconstructed features are useful. Because no persistence, mean-vector, zero-vector, or sparse-feature baseline was evaluated, it is unclear whether the Transformer performs better than simple alternative predictors. The additional nearest-neighbour analysis produced a label score of 46.75% by comparing each predicted feature vector with the feature vectors of all Cora

nodes and assigning the label of the closest match. Although this score is above the majority-class reference, it is not equivalent to clean future-label classification because the candidate pool includes all nodes, including nodes from the test set. The model may therefore retrieve the actual future node or a similar test node during this comparison. The result should be interpreted as a retrieval diagnostic showing that the predicted feature vectors contain some class-related structure, rather than as evidence of reliable unseen-node classification. Overall, Experiment 2 shows that the model can generate future feature vectors, but the practical quality and usefulness of those predictions remain uncertain without stronger baseline comparisons.

Experiment 3 provides the clearest comparison between teacher forcing and generation. During teacher-forced evaluation, the decoder receives the correct previous label at each step. This makes the task easier because an earlier prediction error cannot affect the next input. Under teacher forcing, the model achieved 81.07% token accuracy. During generated evaluation, the decoder must use its own previous predictions, so accuracy fell to 76.76%.

The class-wise results are uneven. Classes 2, 3, and 4 have the strongest F1-scores, while class 0 is harder for both the next-label and generated future-label tasks. The class distribution is imbalanced, so accuracy alone is not sufficient. The macro F1 values of 0.79 for Experiment 1 and 0.75 for generated Experiment 3 provide a more balanced summary.

Experiment 4 is the most important result for interpretation. Although its teacher-forced test token accuracy was 47.93%, generated token accuracy fell to 22.51%, below the approximately 31.67% majority-class baseline. Generated sequence accuracy was only 9.71%. Compared with the main walk-level generated result, this is a drop of approximately 54.25 percentage points in token accuracy and 51.98 percentage points in sequence accuracy. Experiment 4 therefore provides evidence that the high main-split scores do not transfer to official unseen-node partitions.

7 Conclusion

This report presents a Transformer-based sequence approach to the Cora citation graph. Random walks convert local graph structure into sequences, and BERT-style encoders and causal sequence-to-sequence decoders are used for label and feature prediction.

The ordinary walk-level experiments show that the models can learn useful label regularities. The next-label model achieved 81.13% accuracy. The future-label model achieved 81.07% teacher-forced token accuracy and 76.76% generated token accuracy. The future-feature model achieved 0.011614 MSE, 0.2931 cosine similarity, and 46.75% nearest-neighbour label accuracy. These values show that the sequence models capture patterns in the sampled walks, although the feature result still requires a simple reconstruction baseline for proper interpretation.

Experiment 4 provides the central qualification. When Cora’s official train, validation, and test masks were kept separate, generated token accuracy fell to 22.51% and generated sequence accuracy fell to 9.71%. The generated token score was below the 31.67% majority-class baseline. Thus, the strong walk-level performance should not be interpreted as evidence of reliable unseen-node generalisation. Instead, the results indicate a substantial generalisation gap between random-walk evaluation with node overlap and strict node-disjoint evaluation.

Stronger claims would require corrected or extended baselines, repeated seeds, and

investigation of the strict model’s overfitting and class bias.

8 Limitations

1. **Walk overlap in the main experiments.** The main train, validation, and test sets are split by walk, not by node. The same node can therefore occur in multiple partitions.
2. **Single dataset.** Cora is a standard benchmark but represents one citation network and seven classes. Results may not transfer to other graph types.
3. **Single random seed.** The current run uses seed 42. Repeated seeds are needed to estimate variability.
4. **Randomly initialised encoders.** The BERT-style models are not language-pretrained. The experiment tests the architecture, not the benefit of BERT pre-training.
5. **Random-walk stochasticity.** There may be several valid next nodes from the same graph position. Exact next-sequence prediction is therefore an inherently uncertain task.
6. **Sparse feature forecasting.** Cora’s features are binary word vectors. Mean squared error treats all dimensions as numeric targets and may not capture every useful notion of feature similarity.
7. **Nearest-neighbour retrieval pool.** The feature-to-label retrieval diagnostic searches the full Cora feature matrix, including test nodes, and is not a strict held-out classification metric.
8. **No direct GNN baseline.** The study does not yet compare the Transformer sequence models with a GCN, GraphSAGE, or another graph-specific baseline.
9. **Limited hyperparameter search.** The current code uses one main walk configuration and a fixed model configuration rather than a systematic search.
10. **No confidence intervals.** The final report should avoid treating one run as a precise estimate of general performance.
11. **Best-checkpoint restoration.** The current implementation tracks validation loss for early stopping but does not restore the best model state before test evaluation. The Experiment 3 and Experiment 4 test values therefore come from the final stopped epochs rather than the best validation epochs.

9 Future directions

Several extensions would make the comparison stronger. First, the experiment should be repeated over several seeds and report means and standard deviations. Second, node-disjoint splitting should be used as the primary evaluation if the intended claim concerns unseen-node generalisation. Third, simple baselines should be added, including the most

frequent class, a label-transition baseline, feature persistence, and a classifier trained directly on past features.

A graph-aware comparison with GCN, GraphSAGE, or a graph Transformer would show whether the sequence representation provides a practical advantage. The Transformer could also receive node IDs, edge types, or positional information derived from the walk, although these additions would need to be checked carefully for leakage.

The categorical decoder could be evaluated with probabilistic metrics such as negative log-likelihood or calibration rather than only greedy accuracy. Beam search could be compared with greedy decoding, but exact sequence accuracy should still be interpreted cautiously because several future paths may be plausible. Finally, the approach could be tested on graphs with temporal information, where future feature prediction has a more direct meaning than it does for the static Cora benchmark.

References

- [1] Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT*, 2019. <https://arxiv.org/abs/1810.04805>.
- [2] Hamilton, W. L., Ying, R., and Leskovec, J. Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems*, 2017. <https://arxiv.org/abs/1706.02216>.
- [3] Kipf, T. N. and Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. *International Conference on Learning Representations*, 2017. <https://arxiv.org/abs/1609.02907>.
- [4] Li, Q., Han, Z., and Wu, X.-M. Deeper Insights into Graph Convolutional Networks for Semi-Supervised Learning. *Proceedings of AAAI*, 2018. <https://arxiv.org/abs/1801.07606>.
- [5] Loshchilov, I. and Hutter, F. Decoupled Weight Decay Regularization. *International Conference on Learning Representations*, 2019. <https://arxiv.org/abs/1711.05101>.
- [6] Perozzi, B., Al-Rfou, R., and Skiena, S. DeepWalk: Online Learning of Social Representations. *Proceedings of KDD*, 2014. <https://doi.org/10.1145/2623330.2623732>.
- [7] Vaswani, A., Shazeer, N., Parmar, N., et al. Attention Is All You Need. *Advances in Neural Information Processing Systems*, 2017. <https://arxiv.org/abs/1706.03762>.
- [8] Yang, Z., Cohen, W. W., and Salakhutdinov, R. Revisiting Semi-Supervised Learning with Graph Embeddings. *International Conference on Machine Learning*, 2016. <https://arxiv.org/abs/1603.08861>.
- [9] PyTorch Geometric documentation. Planetoid dataset loader and graph utilities. <https://pytorch-geometric.readthedocs.io/>.